

Министерство науки и высшего образования Российской Федерации федеральное
государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Курсовая работа

по дисциплине “Мобильная разработка (Android и iOS)”

по теме “Разработка мобильного приложения “Мессенджер””

Выполнила:

Шемелева Елизавета Сергеевна, К3442

Проверил:

Шатинский Григорий Сергеевич

Санкт-Петербург

2026

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
1 Лабораторная работа 1.....	3
1.1 Задачи и целевые требования.....	3
1.2 Архитектурная организация и примененные технологии.....	3
1.3 Реализация главной активности.....	4
1.4 Разметка главной активности.....	5
1.5 Граф навигации.....	7
1.6 Меню нижней навигационной панели.....	8
1.7 Реализация фрагментов приложения.....	8
1.7.1 Фрагмент новостной ленты.....	9
1.7.2 Фрагмент профиля пользователя.....	10
1.7.3 Фрагмент настроек.....	11
1.8 Отслеживание жизненного цикла компонентов.....	12
1.9 Результаты и выводы.....	13
2 Лабораторная работа 2.....	14
2.1 Задачи и целевые требования.....	14
2.2 Архитектурный паттерн MVVM и его применение.....	14
2.3 Реализация AppViewModel.....	15
2.4 Модель данных UserProfile.....	17
2.5 Фрагмент профиля с реактивным обновлением.....	17
2.6 Фрагмент редактирования профиля.....	19
2.7 Фрагмент настроек с переключателем темы.....	19
2.8 Система тем приложения.....	21
2.9 Сохранение состояния при повороте экрана.....	21
2.10 Результаты и выводы.....	22
3 Лабораторная работа 3.....	23
3.1 Задачи и целевые требования.....	23
3.2 Архитектура трехслойной системы данных.....	23
3.3 Клиент Retrofit и конфигурация API.....	24
3.4 Интерфейс ApiService.....	24
3.5 Модель данных для сетевых ответов.....	25
3.6 Сущность базы данных Room.....	25
3.7 Объект доступа к данным (DAO).....	26
3.8 Конфигурация базы данных Room.....	26
3.9 Проверка доступности сетевого подключения.....	27
3.10 Реализация MessageRepository.....	28

3.11 Обновлённая ViewModel с поддержкой сообщений.....	30
3.12 Фрагмент новостной ленты с RecyclerView.....	30
3.13 Адаптер RecyclerView.....	31
3.14 Разметка экрана новостной ленты.....	32
3.15 Разметка элемента списка.....	34
3.16 Жизненный цикл синхронизации данных.....	35
3.17 Результаты и выводы.....	36
4 Лабораторная работа 4.....	37
4.1 Задачи и требования.....	37
4.2 Обновлённая модель данных.....	37
4.3 Адаптер с функцией лайков.....	38
4.4 Material Design в разметке элемента.....	38
4.5 WorkManager и фоновая синхронизация.....	39
4.6 Уведомления при синхронизации.....	40
4.7 Запрос разрешений и планирование WorkManager.....	41
4.8 Объявление разрешений в манифесте.....	42
4.9 Поток выполнения синхронизации.....	42
4.10 Реактивность интерфейса.....	43
4.11 Результаты и выводы.....	43
ЗАКЛЮЧЕНИЕ.....	44

1 Лабораторная работа 1

1.1 Задачи и целевые требования

Целью первой лабораторной работы была разработка базовой структуры приложения мобильной мессенджер с реализацией многоэкранной навигации. Требовалось выполнить следующие функциональные требования:

- 1) Создать главную активность в качестве единственной точки входа приложения.
- 2) Реализовать систему навигации с использованием компонентов Android Navigation Component (NavController и Navigation Graph).
- 3) Организовать три основных экрана приложения (вкладки) с помощью фрагментов.
- 4) Обеспечить нижнюю навигационную панель (Bottom Navigation) для переключения между экранами.
- 5) Реализовать полное логирование жизненного цикла компонентов для отладки.
- 6) Гарантировать бесперебойный запуск приложения и понятный интерфейс.

Нефункциональные требования включали надлежащее структурирование кода для обеспечения возможности последующего расширения функциональности, игнорирование вспомогательных файлов проекта через файл .gitignore и документирование через систему контроля версий на платформе GitHub.

1.2 Архитектурная организация и примененные технологии

При реализации проекта были использованы следующие основные компоненты фреймворка Android:

- 1) Android Navigation Component – для декларативного управления переходами между экранами.

- 2) `Fragment` – для создания переиспользуемых компонентов пользовательского интерфейса.
- 3) `ConstraintLayout` – для адаптивного и гибкого размещения элементов интерфейса.
- 4) `BottomNavigationView` – компонент материального дизайна для организации нижней навигационной панели.
- 5) `Kotlin` – язык программирования для разработки на платформе `Android`.

1.3 Реализация главной активности

Главная активность (`MainActivity`) выступает в роли контейнера для всех фрагментов и обеспечивает интеграцию навигационной системы. В методе `onCreate()` инициализируется `NavHostFragment`, который служит контейнером для размещения фрагментов, и осуществляется связь с нижней навигационной панелью.

Листинг кода – `MainActivity.kt`:

```
class MainActivity : AppCompatActivity() {
    private val TAG = "LifeCycleLog"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        Log.d(TAG, "MainActivity: onCreate()")

        // Получение контроллера навигации из фрагмента
        val navHostFragment = supportFragmentManager
            .findFragmentById(R.id.nav_host_fragment) as
NavHostFragment
        val navController = navHostFragment.navController

        // Связь нижней панели навигации с контроллером
        val bottomNav =
findViewById<BottomNavigationView>(R.id.bottom_nav)
        bottomNav.setupWithNavController(navController)
    }

    // Логирование жизненного цикла активности
    override fun onStart() {
        super.onStart()
    }
}
```

```

        Log.d(TAG, "MainActivity: onStart()")
    }

    override fun onResume() {
        super.onResume()
        Log.d(TAG, "MainActivity: onResume()")
    }

    override fun onPause() {
        super.onPause()
        Log.d(TAG, "MainActivity: onPause()")
    }

    override fun onStop() {
        super.onStop()
        Log.d(TAG, "MainActivity: onStop()")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "MainActivity: onDestroy()")
    }
}

```

1.4 Разметка главной активности

Разметка главной активности организована с использованием `ConstraintLayout` и включает два основных компонента:

- 1) `NavHostFragment` — контейнер для размещения фрагментов;
- 2) `BottomNavigationView` — нижняя панель навигации.

Листинг кода – `activity_main.xml`:

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/nav_host_fragment"

        android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toTopOf="@+id/bottom_nav"
        app:layout_constraintStart_toStartOf="parent"

```

```

        app:layout_constraintEnd_toEndOf="parent"
        app:navGraph="@navigation/nav_graph"
        app:defaultNavHost="true" />

<com.google.android.material.bottomnavigation.BottomNavigationView
    android:id="@+id/bottom_nav"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:menu="@menu/bottom_nav_menu" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

NavHostFragment автоматически управляет стеком фрагментов в соответствии с определённым навигационным графом. Атрибут `app:navGraph` указывает на файл описания навигационной структуры.

1.5 Граф навигации

Граф навигации (Navigation Graph) представляет собой декларативное описание структуры переходов между экранами приложения. Он определяет, какие фрагменты доступны, какие действия между ними возможны и какой из них является начальным.

Листинг кода – nav_graph.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<navigation
xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  android:id="@+id/nav_graph"
  app:startDestination="@id/newsFeedFragment">

  <fragment
    android:id="@+id/newsFeedFragment"
    android:name="com.example.messenger.NewsFeedFragment"
    android:label="Новости" />

  <fragment
    android:id="@+id/profileFragment"
    android:name="com.example.messenger.ProfileFragment"
    android:label="Профиль">
    <action
      android:id="@+id/action_profileFragment_to_editProfileFragment"
      app:destination="@id/editProfileFragment" />
    </fragment>

  <fragment
    android:id="@+id/editProfileFragment"
    android:name="com.example.messenger.EditProfileFragment"
    android:label="Редактировать профиль" />

  <fragment
    android:id="@+id/settingsFragment"
    android:name="com.example.messenger.SettingsFragment"
    android:label="Настройки" />

</navigation>
```

Атрибут `app:startDestination` указывает начальный фрагмент, который загружается при запуске приложения. Элементы `<action>` описывают возможные переходы между фрагментами и используются для навигации в коде.

1.6 Меню нижней навигационной панели

Меню нижней панели определяет элементы, которые отображаются в навигационной полосе. Каждый элемент соответствует одному из фрагментов основного приложения.

Листинг кода – bottom_nav_menu.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<menu
xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/newsFeedFragment"
        android:icon="@drawable/nav_feed_selector"
        android:title="Новости" />

    <item
        android:id="@+id/profileFragment"
        android:icon="@drawable/nav_profile_selector"
        android:title="Профиль" />

    <item
        android:id="@+id/settingsFragment"
        android:icon="@drawable/nav_settings_selector"
        android:title="Настройки" />

</menu>
```

Идентификаторы элементов меню (атрибут android:id) совпадают с идентификаторами фрагментов в графе навигации, что позволяет система автоматически управлять переходами при выборе пользователем пункта меню.

1.7 Реализация фрагментов приложения

Приложение включает три основных фрагмента: экран новостной ленты, экран профиля пользователя и экран настроек.

1.7.1 Фрагмент новостной ленты

Фрагмент новостной ленты (NewsFeedFragment) выступает в роли начального экрана приложения. На данном этапе разработки он содержит только логирование жизненного цикла.

Листинг кода – NewsFeedFragment.kt:

```
class NewsFeedFragment : Fragment(R.layout.fragment_news_feed) {
    private val TAG = "LifeCycleLog"

    override fun onAttach(context: Context) {
        super.onAttach(context)
        Log.d(TAG, "NewsFeedFragment: onAttach() - Присоединение
к Activity")
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "NewsFeedFragment: onCreate()")
    }

    override fun onCreateView(view: View, savedInstanceState:
Bundle?) {
        super.onCreateView(view, savedInstanceState)
        Log.d(TAG, "NewsFeedFragment: onCreateView() - Разметка
создана")
    }

    override fun onStart() {
        super.onStart()
        Log.d(TAG, "NewsFeedFragment: onStart() - Фрагмент
виден")
    }

    override fun onResume() {
        super.onResume()
        Log.d(TAG, "NewsFeedFragment: onResume() - Фрагмент
активен")
    }

    override fun onPause() {
        super.onPause()
        Log.d(TAG, "NewsFeedFragment: onPause() - Фрагмент
теряет фокус")
    }

    override fun onStop() {
        super.onStop()
    }
}
```

```

        Log.d(TAG, "NewsFeedFragment: onStop() - Фрагмент не
виден")
    }

    override fun onDestroyView() {
        super.onDestroyView()
        Log.d(TAG, "NewsFeedFragment: onDestroyView() - Разметка
уничтожена")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "NewsFeedFragment: onDestroy() - Фрагмент
уничтожен")
    }

    override fun onDetach() {
        super.onDetach()
        Log.d(TAG, "NewsFeedFragment: onDetach() - Отсоединение
от Activity")
    }
}

```

1.7.2 Фрагмент профиля пользователя

Фрагмент профиля (ProfileFragment) отображает основные данные пользователя и предоставляет доступ к экрану редактирования профиля через навигационное действие.

Листинг кода – ProfileFragment.kt:

```

class ProfileFragment : Fragment(R.layout.fragment_profile) {
    private val TAG = "LifeCycleLog"
    private lateinit var nameTextView: TextView
    private lateinit var usernameTextView: TextView
    private lateinit var phoneTextView: TextView

    private fun loadProfileData() {
        nameTextView.text = ProfileData.user.name
        usernameTextView.text = "@${ProfileData.user.username}"
        phoneTextView.text = ProfileData.user.phoneNumber
    }

    override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        Log.d(TAG, "ProfileFragment: onViewCreated() - Разметка
создана")
    }
}

```

```

                                val editButton =
view.findViewById<View>(R.id.tv_edit_button)
    nameTextView = view.findViewById(R.id.tv_user_name)
    usernameTextView = view.findViewById(R.id.tv_username)
    phoneTextView = view.findViewById(R.id.tv_phone_number)

    editButton.setOnClickListener {

findNavController().navigate(R.id.editProfileFragment)
    }

    loadProfileData()
}

override fun onResume() {
    super.onResume()
    Log.d(TAG, "ProfileFragment: onResume() - Фрагмент
активен")
    loadProfileData()
}

// Остальные методы жизненного цикла с логированием...
}

```

1.7.3 Фрагмент настроек

Фрагмент настроек (SettingsFragment) предоставляет минимальный функционал интерфейса настроек, необходимый для выполнения требований задания.

Листинг кода – SettingsFragment.kt:

```

class SettingsFragment : Fragment(R.layout.fragment_settings) {
    private val TAG = "LifeCycleLog"
    private lateinit var themeSwitcherImageView: ImageView

    override fun onCreateView(view: View, savedInstanceState:
Bundle?) {
        super.onCreateView(view, savedInstanceState)
        Log.d(TAG, "SettingsFragment: onCreateView() - Разметка
создана")

                                themeSwitcherImageView =
view.findViewById(R.id.iv_theme_switcher)
        updateThemeSwitcherIcon()

        themeSwitcherImageView.setOnClickListener {
            toggleTheme()
        }
    }
}

```

```

        private fun toggleTheme() {
            val currentNightMode =
AppCompatActivity.getDefaultNightMode()
            val newNightMode = if (currentNightMode ==
AppCompatActivity.MODE_NIGHT_YES) {
                AppCompatActivity.MODE_NIGHT_NO
            } else {
                AppCompatActivity.MODE_NIGHT_YES
            }

            AppCompatActivity.setDefaultNightMode(newNightMode)
            activity?.recreate()
        }

        private fun updateThemeSwitcherIcon() {
            val isNightMode =
AppCompatActivity.getDefaultNightMode()
AppCompatActivity.MODE_NIGHT_YES
            if (isNightMode) {

themeSwitcherImageView.setImageResource(R.drawable.ic_sun)
            } else {

themeSwitcherImageView.setImageResource(R.drawable.ic_moon)
            }
        }

        // Остальные методы жизненного цикла с логированием...
    }

```

1.8 Отслеживание жизненного цикла компонентов

Во всех компонентах приложения реализовано полное логирование методов жизненного цикла. Это необходимо для отладки и понимания последовательности инициализации и завершения компонентов. При каждом методе жизненного цикла выполняется запись в журнал отладки (Log) с использованием тега `LifeCycleLog` для удобства фильтрации в инструменте Logcat.

Последовательность вызовов методов жизненного цикла для фрагмента следующая:

- 1) `onAttach()` – фрагмент присоединяется к активности;
- 2) `onCreate()` – инициализация фрагмента;

- 3) `onViewCreated()` – разметка (макет) фрагмента создана и готова к использованию;
- 4) `onStart()` – фрагмент становится видимым;
- 5) `onResume()` – фрагмент становится активным и взаимодействует с пользователем;
- 6) `onPause()` – фрагмент теряет фокус;
- 7) `onStop()` – фрагмент больше не видим;
- 8) `onDestroyView()` – разметка фрагмента удаляется;
- 9) `onDestroy()` – фрагмент удаляется;
- 10) `onDetach()` – фрагмент отсоединяется от активности.

1.9 Результаты и выводы

По завершении первой лабораторной работы было создано функциональное приложение с полной реализацией многоэкранной навигации. Система навигации на основе `Navigation Component` позволила организовать переходы между экранами декларативным способом, что способствует улучшению читаемости кода и упрощает управление навигационными логикой.

Логирование жизненного цикла компонентов продемонстрировало корректное взаимодействие фрагментов с основной активностью и помогло выявить порядок инициализации и завершения компонентов. Приложение успешно запускается без ошибок, интерфейс понятен, а структура кода пригодна для последующего расширения функциональности.

2 Лабораторная работа 2

2.1 Задачи и целевые требования

Целью второй лабораторной работы была реализация архитектурного паттерна MVVM (Model-View-ViewModel) для управления состоянием приложения и обеспечения сохранения пользовательских данных при изменении конфигурации устройства (например, при повороте экрана). Требовалось выполнить следующие функциональные требования:

- 1) Создать компонент ViewModel для хранения состояния профиля и настроек приложения.
- 2) На экране профиля реализовать редактируемые поля имени и статуса пользователя с сохранением данных во ViewModel.
- 3) На экране настроек реализовать переключатель темы (светлая/тёмная) с сохранением состояния во ViewModel.
- 4) Гарантировать, что при повороте экрана данные не теряются.
- 5) Использовать механизм LiveData для реактивного обновления интерфейса.
- 6) Реализовать логирование этапов жизненного цикла ViewModel.

Нефункциональные требования предусматривали разделение ответственности между слоем представления (View/Fragment) и слоем логики (ViewModel), а также обеспечение возможности последующего расширения функциональности приложения.

2.2 Архитектурный паттерн MVVM и его применение

MVVM (Model-View-ViewModel) — это архитектурный паттерн, предназначенный для разделения ответственности между пользовательским интерфейсом и логикой приложения:

- 1) View – фрагменты и активности, отвечающие за отображение данных пользователю.

- 2) ViewModel – компонент, хранящий логику управления состоянием и обеспечивающий данные для представления.
- 3) Model – структуры данных, сохраняемые в локальном хранилище (SharedPreferences) или в других источниках данных.

Применение этого паттерна гарантирует, что состояние приложения сохраняется при изменении конфигурации устройства и обеспечивает чистоту архитектуры кода.

2.3 Реализация AppViewModel

AppViewModel выступает в роли центрального хранилища состояния приложения. Она расширяет класс AndroidViewModel, что позволяет ей получать доступ к контексту приложения и использовать локальное хранилище (SharedPreferences) для сохранения данных.

Листинг кода – AppViewModel.kt:

```
import android.app.Application
import android.content.Context
import android.util.Log
import androidx.lifecycle.AndroidViewModel
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData

class AppViewModel(application: Application) :
    AndroidViewModel(application) {

    private val TAG = "LifeCycleLog"
    private val PREFS_NAME = "profile_prefs"
    private val prefs =
        application.getSharedPreferences(PREFS_NAME,
            Context.MODE_PRIVATE)

    private val KEY_NAME = "name"
    private val KEY_USERNAME = "username"
    private val KEY_PHONE = "phoneNumber"
    private val KEY_DARK_MODE = "isDarkMode"

    // Данные профиля пользователя
    private val _userProfile = MutableLiveData(loadProfile())
    val userProfile: LiveData<UserProfile> = _userProfile

    // Состояние темы
```

```

private val _isDarkMode = MutableLiveData(loadDarkMode())
val isDarkMode: LiveData<Boolean> = _isDarkMode

init {
    Log.d(TAG, "AppViewModel: init - Создан")
}

// Загрузка профиля из SharedPreferences
private fun loadProfile(): UserProfile {
    return UserProfile(
        name = prefs.getString(KEY_NAME, "Laura Palmer") ?:
"Laura Palmer",
        username = prefs.getString(KEY_USERNAME,
"meanwhile") ?: "meanwhile",
        phoneNumber = prefs.getString(KEY_PHONE, "+0 (000)
000-00-00") ?: "+0 (000) 000-00-00"
    )
}

// Сохранение профиля в ViewModel и SharedPreferences
fun saveProfile(name: String, username: String, phoneNumber:
String) {
    _userProfile.value = UserProfile(name, username,
phoneNumber)
    prefs.edit().apply {
        putString(KEY_NAME, name)
        putString(KEY_USERNAME, username)
        putString(KEY_PHONE, phoneNumber)
        apply()
    }
}

// Загрузка состояния темы из SharedPreferences
private fun loadDarkMode(): Boolean {
    return prefs.getBoolean(KEY_DARK_MODE, false)
}

// Установка состояния темы
fun setDarkMode(enabled: Boolean) {
    if (_isDarkMode.value != enabled) {
        _isDarkMode.value = enabled
        prefs.edit().putBoolean(KEY_DARK_MODE,
enabled).apply()
    }
}

// Логирование при удалении ViewModel
override fun onCleared() {
    super.onCleared()
    Log.d(TAG, "AppViewModel: onCleared - Уничтожен")
}
}

```

Ключевые особенности реализации:

- 1) `MutableLiveData` используется для хранения изменяемого состояния, которое может быть обновлено из различных слоёв приложения.
- 2) `Livedata` предоставляется как публичное свойство, что гарантирует, что фрагменты могут подписываться на изменения, но не могут напрямую изменять данные.
- 3) `SharedPreferences` используется для персистентного сохранения данных между сеансами приложения.
- 4) Метод `onCleared()` вызывается системой при уничтожении `ViewModel` и используется для очистки ресурсов.

2.4 Модель данных `UserProfile`

Модель данных представляет собой простую структуру данных (`data class`) для хранения информации о профиле пользователя:

```
data class UserProfile(  
    val name: String,  
    val username: String,  
    val phoneNumber: String  
)
```

Использование `data class` в Kotlin автоматически генерирует методы `equals()`, `hashCode()` и `toString()`, что упрощает работу с объектами.

2.5 Фрагмент профиля с реактивным обновлением

Фрагмент профиля (`ProfileFragment`) использует механизм `Livedata` для автоматического обновления интерфейса при изменении данных в `ViewModel`.

Листинг кода – `ProfileFragment.kt`:

```
import android.content.Context  
import android.os.Bundle  
import android.util.Log  
import android.view.View  
import android.widget.TextView  
import androidx.fragment.app.Fragment  
import androidx.fragment.app.activityViewModels
```

```

import androidx.navigation.fragment.findNavController

class ProfileFragment : Fragment(R.layout.fragment_profile) {

    private val TAG = "LifeCycleLog"
    private val viewModel: AppViewModel by activityViewModels()
    private lateinit var nameTextView: TextView
    private lateinit var usernameTextView: TextView
    private lateinit var phoneTextView: TextView

    override fun onAttach(context: Context) {
        super.onAttach(context)
        Log.d(TAG, "ProfileFragment: onAttach() - Присоединение
к Activity")
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "ProfileFragment: onCreate()")
    }

    override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        Log.d(TAG, "ProfileFragment: onViewCreated() - Разметка
создана")

        val editButton =
view.findViewById<View>(R.id.tv_edit_button)
        nameTextView = view.findViewById(R.id.tv_user_name)
        usernameTextView = view.findViewById(R.id.tv_username)
        phoneTextView = view.findViewById(R.id.tv_phone_number)

        editButton.setOnClickListener {

findNavController().navigate(R.id.editProfileFragment)
        }

        // Подписка на изменения профиля через LiveData
        viewModel.userProfile.observe(viewLifecycleOwner) {
profile ->
            nameTextView.text = profile.name
            usernameTextView.text = "@${profile.username}"
            phoneTextView.text = profile.phoneNumber
        }
    }

    override fun onStart() {
        super.onStart()
        Log.d(TAG, "ProfileFragment: onStart() - Фрагмент
виден")
    }
}

```

```
    // Остальные методы жизненного цикла с логированием  
}
```

Метод `observe()` регистрирует наблюдателя, который автоматически вызывается при каждом изменении данных в `LiveData`. Параметр `viewLifecycleOwner` обеспечивает автоматическую отписку от изменений при уничтожении представления фрагмента.

2.6 Фрагмент редактирования профиля

Фрагмент редактирования профиля позволяет пользователю изменять информацию о профиле. При нажатии кнопки сохранения данные отправляются в `ViewModel` и сохраняются в `SharedPreferences`. Это гарантирует, что изменения будут видны на экране профиля благодаря реактивному обновлению через `LiveData`.

Фрагмент включает следующие операции:

- 1) Загрузку текущего профиля из `ViewModel`.
- 2) Редактирование полей имени, имени пользователя и номера телефона.
- 3) Валидацию входных данных перед сохранением.
- 4) Вызов метода `saveProfile()` `ViewModel` для сохранения данных.
- 5) Навигацию обратно на экран профиля после успешного сохранения.

2.7 Фрагмент настроек с переключателем темы

Фрагмент настроек (`SettingsFragment`) реализует переключатель темы приложения. При нажатии кнопки переключения тема меняется, и состояние сохраняется в `ViewModel`.

Листинг кода – `SettingsFragment.kt`:

```
import android.os.Bundle  
import android.util.Log  
import android.util.TypedValue
```

```

import android.view.View
import android.widget.ImageView
import androidx.appcompat.app.AppCompatActivity
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels

class SettingsFragment : Fragment(R.layout.fragment_settings) {

    private val TAG = "LifeCycleLog"
    private lateinit var themeSwitcherImageView: ImageView
    private val viewModel: AppViewModel by activityViewModels()

    override fun onCreateView(view: View, savedInstanceState: Bundle?) {
        super.onCreateView(view, savedInstanceState)
        Log.d(TAG, "SettingsFragment: onCreateView() - Разметка создана")

        themeSwitcherImageView =
view.findViewById(R.id.iv_theme_switcher)

        // Подписка на изменения состояния темы
        viewModel.isDarkMode.observe(viewLifecycleOwner) {
isDark ->
            val iconRes = if (isDark) R.drawable.ic_moon else
R.drawable.ic_sun
            themeSwitcherImageView.setImageResource(iconRes)

            // Применение цвета значка в соответствии с темой
            val typedValue = TypedValue()
            val theme = requireContext().theme
            theme.resolveAttribute(

com.google.android.material.R.attr.colorOnBackground,
                typedValue,
                true
            )
            val color = typedValue.data
            themeSwitcherImageView.setColorFilter(color)
        }

        themeSwitcherImageView.setOnClickListener {
            toggleTheme()
        }
    }

    private fun toggleTheme() {
        val currentIsDark = viewModel.isDarkMode.value ?: false
        val newIsDark = !currentIsDark

        viewModel.setDarkMode(newIsDark)
    }
}

```

```

        val mode = if (newIsDark)
            AppCompatActivity.MODE_NIGHT_YES
        else
            AppCompatActivity.MODE_NIGHT_NO

        if (AppCompatActivity.getDefaultNightMode() != mode) {
            AppCompatActivity.setDefaultNightMode(mode)
        }
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "SettingsFragment: onCreate()")
    }

    override fun onStart() {
        super.onStart()
        Log.d(TAG, "SettingsFragment: onStart() - Фрагмент
виден")
    }

    // Остальные методы
}

```

2.8 Система тем приложения

Для поддержки светлой и тёмной тем приложения используются два файла конфигурации тем: один для светлой темы (по умолчанию) и один для тёмной темы (в каталоге `values-night`).

Система автоматически выбирает подходящую тему на основе состояния темы устройства и значения, установленного в `ViewModel`.

2.9 Сохранение состояния при повороте экрана

Одна из основных преимуществ использования `ViewModel` и `LiveData` заключается в том, что `ViewModel` не уничтожается при изменении конфигурации устройства (например, при повороте экрана). Это означает, что все данные остаются в памяти и доступны при пересоздании фрагмента.

Последовательность событий при повороте экрана:

- 1) `onPause()` и `onStop()` вызываются на текущем фрагменте.
- 2) `onDestroyView()` удаляет старый макет представления.

- 3) ViewModel остаётся в памяти и сохраняет свои данные.
- 4) onCreate() и onCreateView() создают новый макет представления с тем же ViewModel.

Наблюдатели (observers) подписываются на LiveData, и интерфейс обновляется с сохраненным состоянием.

2.10 Результаты и выводы

По завершении второй лабораторной работы было реализовано управление состоянием приложения с использованием архитектурного паттерна MVVM. Использование ViewModel и LiveData обеспечило реактивное обновление интерфейса и сохранение состояния при изменении конфигурации устройства.

Система тем приложения позволила пользователям выбирать между светлой и тёмной темами, а состояние выбора сохраняется в локальном хранилище. Разделение ответственности между слоем представления и слоем логики упростило управление кодом и обеспечило возможность последующего расширения функциональности приложения без изменения структуры архитектуры.

3 Лабораторная работа 3

3.1 Задачи и целевые требования

Целью третьей лабораторной работы была реализация загрузки данных из внешнего API и организация локального кэширования с поддержкой работы в условиях отсутствия сетевого подключения. Требовалось выполнить следующие функциональные требования:

- 1) Создать интерфейс ApiService для взаимодействия с публичным API через библиотеку Retrofit.
- 2) Реализовать компонент MessageRepository как единственную точку взаимодействия ViewModel с источниками данных.
- 3) Разработать локальную базу данных с использованием фреймворка Room для сохранения сообщений.
- 4) Отобразить список сообщений на вкладке лента с помощью компонента RecyclerView.
- 5) Реализовать возможность ручного обновления данных кнопкой «Обновить» и механизмом pull-to-refresh.
- 6) При отсутствии сетевого подключения загружать сообщения из локальной базы данных.

Нефункциональные требования предусматривали использование корутин для асинхронных сетевых операций, применение паттерна Repository для обслуживания данных и реактивное обновление интерфейса через LiveData.

3.2 Архитектура трехслойной системы данных

Реализованная архитектура следует классическому паттерну трехслойной системы, состоящей из трех основных компонентов:

- 1) API слой – удаленный источник данных (REST сервис через Retrofit).
- 2) Слой локального хранилища – локальная база данных (Room).

3) Repository слой – бизнес-логика управления данными.

Такая организация обеспечивает гибкость, тестируемость и легкость замены источников данных при необходимости.

3.3 Клиент Retrofit и конфигурация API

Для взаимодействия с удалённым API используется библиотека Retrofit, которая предоставляет удобный способ определения HTTP-операций.

Листинг кода – RetrofitClient.kt:

```
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor

object RetrofitClient {
    private const val BASE_URL =
    "https://jsonplaceholder.typicode.com/"

    // Интерцептор для логирования HTTP-запросов и ответов
    private val logging = HttpLoggingInterceptor().apply {
        level = HttpLoggingInterceptor.Level.BODY
    }

    // Конфигурация HTTP-клиента
    private val client = OkHttpClient.Builder()
        .addInterceptor(logging)
        .build()

    val apiService: ApiService by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .client(client)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(ApiService::class.java)
    }
}
```

3.4 Интерфейс ApiService

Интерфейс ApiService определяет методы для получения данных из удаленного API.

Листинг кода — ApiService.kt:

```
import retrofit2.http.GET

interface ApiService {
    @GET("posts")
    suspend fun getPosts(): List<MessageResponse>
}
```

Ключевые элементы:

- 1) `@GET("posts")` – аннотация, определяющая GET-запрос к эндпоинту `/posts`.
- 2) `suspend` – модификатор, указывающий, что функция является сопрограммой и может быть приостановлена без блокировки потока.

3.5 Модель данных для сетевых ответов

Модель `MessageResponse` отображает структуру ответа от API и используется `Gson` для десериализации JSON.

Листинг кода – `MessageResponse.kt`:

```
data class MessageResponse(
    val id: Int,
    val title: String,
    val body: String,
    val userId: Int
)
```

3.6 Сущность базы данных Room

Сущность `Message` определяет структуру таблицы в локальной базе данных и включает дополнительные аннотации для конфигурации Room.

Листинг кода – `Message.kt`:

```
import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "messages")
data class Message(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0,
    val title: String,
    val body: String?,
    val userId: Int
)
```

Аннотация `@Entity(tableName = "messages")` указывает, что класс представляет таблицу в базе данных, а `@PrimaryKey(autoGenerate = true)` определяет поле автоинкрементирующегося первичного ключа.

3.7 Объект доступа к данным (DAO)

Интерфейс `MessageDao` содержит методы для взаимодействия с таблицей сообщений в базе данных.

Листинг кода – `MessageDao.kt`:

```
import androidx.lifecycle.LiveData
import androidx.room.*

@Dao
interface MessageDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertMessages(messages: List<Message>)

    @Query("SELECT * FROM messages ORDER BY id DESC")
    fun getAllMessages(): LiveData<List<Message>>

    @Query("DELETE FROM messages")
    suspend fun deleteAllMessages()
}
```

Методы аннотируются следующим образом:

- 1) `@Insert` – вставка новых записей с указанной стратегией разрешения конфликтов.
- 2) `@Query` – выполнение произвольных SQL-запросов, в том числе выборку и удаление.
- 3) `suspend` – обозначает асинхронные операции, которые должны выполняться в корутинах.

3.8 Конфигурация базы данных Room

Класс `AppDatabase` определяет конфигурацию базы данных и предоставляет доступ к DAO-объектам.

Листинг кода – `AppDatabase.kt`:

```
import android.content.Context
```

```

import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(entities = [Message::class], version = 1, exportSchema
= false)
abstract class AppDatabase : RoomDatabase() {

    abstract fun messageDao(): MessageDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "messenger_database"
                ).build()

                INSTANCE = instance
                instance
            }
        }
    }
}

```

Реализован паттерн singleton с потокобезопасностью (synchronized) для гарантирования единственного экземпляра базы данных на всём протяжении работы приложения.

3.9 Проверка доступности сетевого подключения

Утилита NetworkUtils предоставляет методы для проверки наличия активного сетевого подключения.

Листинг кода – NetworkUtils.kt:

```

import android.content.Context
import android.net.ConnectivityManager
import android.net.NetworkCapabilities

object NetworkUtils {
    fun isNetworkAvailable(context: Context): Boolean {
        val connectivityManager = context.getSystemService(
            Context.CONNECTIVITY_SERVICE
        ) as ConnectivityManager
    }
}

```

```

        val network = connectivityManager.activeNetwork ?:
return false
        val capabilities =
connectivityManager.getNetworkCapabilities(network) ?: return
false
        return
capabilities.hasCapability(NetworkCapabilities.NET_CAPABILITY_IN
TERNET)
    }
}

```

Функция проверяет наличие активной сетевой сети и наличие возможности подключения к интернету с использованием современного API (NetworkCapabilities).

3.10 Реализация MessageRepository

MessageRepository выступает в роли единственной точки взаимодействия с данными и реализует логику выбора между сетевым API и локальной базой данных.

Листинг кода – MessageRepository.kt:

```

import android.content.Context
import androidx.lifecycle.LiveData
import androidx.lifecycle.LiveData
import kotlinx.coroutines.Dispatchers

class MessageRepository(private val context: Context) {

    private val messageDao =
AppDatabase.getDatabase(context).messageDao()
    private val apiService = RetrofitClient.apiService

    // Получение сообщений с поддержкой offline-режима
    fun getMessages(): LiveData<List<Message>> =
liveData(Dispatchers.IO) {
        try {
            // Если есть сетевое соединение, загружаем с API
            if (NetworkUtils.isNetworkAvailable(context)) {
                val apiMessages = apiService.getPosts()
                val messages = apiMessages.map {
                    Message(
                        id = it.id.toLong(),
                        title = it.title,
                        body = it.body,
                        userId = it.userId
                    )
                }
            }
        }
    }
}

```

```

        }

        // Очищаем и сохраняем новые данные
        messageDao.deleteAllMessages()
        messageDao.insertMessages(messages)
    }
} catch (e: Exception) {
    e.printStackTrace()
}

// Выдаём список из базы данных (или пустой список, если
база пуста)
emitSource(messageDao.getAllMessages())
}

// Ручное обновление с API
suspend fun refreshMessages() {
    try {
        if (NetworkUtils.isNetworkAvailable(context)) {
            val apiMessages = apiService.getPosts()
            val messages = apiMessages.map {
                Message(
                    id = it.id.toLong(),
                    title = it.title,
                    body = it.body,
                    userId = it.userId
                )
            }

            messageDao.deleteAllMessages()
            messageDao.insertMessages(messages)
        }
    } catch (e: Exception) {
        e.printStackTrace()
    }
}
}

```

Ключевые особенности реализации:

- 1) `liveData()` строитель – создаёт `LiveData`, которая выполняет код на фоновом потоке (`Dispatchers.IO`).
- 2) `emitSource()` – направляет результаты из DAO в наблюдателя.
- 3) Обработка ошибок – все исключения логируются, приложение продолжает работу с кэшированными данными.
- 4) `Offline-first` подход – сначала проверяется сетевое соединение, затем отправляются данные из локальной базы.

3.11 Обновлённая ViewModel с поддержкой сообщений

AppViewModel расширена для управления списком сообщений с использованием Repository.

```
class AppViewModel(application: Application) :  
    AndroidViewModel(application) {  
    // ... остальной код из Lab 2 ...  
  
    private val messageRepository =  
        MessageRepository(application)  
  
    val messages: LiveData<List<Message>> =  
        messageRepository.getMessages()  
  
    fun refreshMessages() {  
        viewModelScope.launch {  
            messageRepository.refreshMessages()  
        }  
    }  
}
```

Использование viewModelScope обеспечивает автоматическую отмену корутин при уничтожении ViewModel.

3.12 Фрагмент новостной ленты с RecyclerView

Фрагмент новостной ленты отображает список сообщений и обеспечивает возможность ручного обновления.

Листинг кода – NewsFeedFragment.kt:

```
import android.content.Context  
import android.os.Bundle  
import android.util.Log  
import android.view.View  
import android.widget.Button  
import android.widget.TextView  
import androidx.fragment.app.Fragment  
import androidx.fragment.app.activityViewModels  
import androidx.recyclerview.widget.RecyclerView  
import androidx.swiperefreshlayout.widget.SwipeRefreshLayout  
  
class NewsFeedFragment : Fragment(R.layout.fragment_news_feed) {  
  
    private val TAG = "LifeCycleLog"  
    private val viewModel: AppViewModel by activityViewModels()  
    private lateinit var recyclerView: RecyclerView  
    private lateinit var adapter: MessageAdapter
```

```

private lateinit var swipeRefreshLayout: SwipeRefreshLayout
private lateinit var btnRefresh: Button
private lateinit var tvPlaceholder: TextView

override fun onCreateView(view: View, savedInstanceState:
Bundle?) {
    super.onCreateView(view, savedInstanceState)
    Log.d(TAG, "NewsFeedFragment: onCreateView() - Разметка
создана")

    recyclerView = view.findViewById(R.id.rv_messages)
    swipeRefreshLayout =
view.findViewById(R.id.swipe_refresh_layout)
    btnRefresh = view.findViewById(R.id.btn_refresh)
    tvPlaceholder =
view.findViewById(R.id.tv_feed_placeholder)

    adapter = MessageAdapter()
    recyclerView.adapter = adapter

    // Подписка на изменения списка сообщений
    viewModel.messages.observe(viewLifecycleOwner) {
messages ->
        adapter.updateMessages(messages)
        tvPlaceholder.visibility = if (messages.isEmpty())
View.VISIBLE else View.GONE
        swipeRefreshLayout.isRefreshing = false
    }

    // Pull-to-refresh механизм
    swipeRefreshLayout.setOnRefreshListener {
        viewModel.refreshMessages()
    }

    // Кнопка ручного обновления
    btnRefresh.setOnClickListener {
        swipeRefreshLayout.isRefreshing = true
        viewModel.refreshMessages()
    }
}

// Остальные методы
}

```

3.13 Адаптер RecyclerView

Адаптер MessageAdapter управляет отображением элементов списка сообщений.

Листинг кода – MessageAdapter.kt:

```
import android.view.LayoutInflater
```

```

import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView

class MessageAdapter :
RecyclerView.Adapter<MessageAdapter.MessageViewHolder>() {

    private var messages: List<Message> = emptyList()

    inner class MessageViewHolder(itemView: View) :
RecyclerView.ViewHolder(itemView) {
        private val titleTextView: TextView =
itemView.findViewById(R.id.tvTitle)
        private val bodyTextView: TextView =
itemView.findViewById(R.id.tvBody)
        private val userIdTextView: TextView =
itemView.findViewById(R.id.tvUserId)

        fun bind(message: Message) {
            titleTextView.text = message.title
            bodyTextView.text = message.body ?: "Нет текста"
            userIdTextView.text = "User ID: ${message.userId}"
        }
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType:
Int): MessageViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_message, parent, false)
        return MessageViewHolder(view)
    }

    override fun onBindViewHolder(holder: MessageViewHolder,
position: Int) {
        holder.bind(messages[position])
    }

    override fun getItemCount(): Int = messages.size

    fun updateMessages(newMessages: List<Message>) {
        messages = newMessages
        notifyDataSetChanged()
    }
}

```

3.14 Разметка экрана новостной ленты

Листинг кода – fragment_news_feed.xml:

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"

```

```

xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:background="?android:attr/colorBackground"
tools:context=".NewsFeedFragment">

```

```

<TextView
    android:id="@+id/tv_feed_title"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:fontFamily="@font/cygre_semibold"
    android:paddingStart="17dp"
    android:paddingTop="33dp"
    android:paddingBottom="12dp"
    android:text="@string/title_news_feed"
    android:textColor="?attr/colorOnBackground"
    android:textSize="25sp"
    android:textStyle="bold"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

```

```

<Button
    android:id="@+id/btn_refresh"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginEnd="17dp"
    android:layout_marginTop="33dp"
    android:text="Обновить"
    android:textSize="14sp"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

```

```

<androidx.swiperefreshlayout.widget.SwipeRefreshLayout
    android:id="@+id/swipe_refresh_layout"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_marginTop="12dp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/tv_feed_title">

```

```

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rv_messages"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"

```

```

app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager"
    >

```

```

        tools:listitem="@layout/item_message" />

</androidx.swiperefreshlayout.widget.SwipeRefreshLayout>

<TextView
    android:id="@+id/tv_feed_placeholder"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:fontFamily="@font/cygre_book"
    android:text="@string/placeholder_empty"
    android:textColor="@color/dark_grey"
    android:textSize="20sp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

Разметка включает:

- 1) RecyclerView – контейнер для отображения списка сообщений.
- 2) SwipeRefreshLayout – обёртка для реализации механизма pull-to-refresh.
- 3) Button – кнопка для ручного обновления списка.
- 4) Placeholder TextView – заголовок, отображаемый при отсутствии сообщений.

3.15 Разметка элемента списка

Листинг кода – item_message.xml:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="12dp">

    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textStyle="bold"
        android:textSize="16sp"
        android:textColor="@android:color/black"

```

```

        android:maxLines="1"
        android:ellipsize="end" />

<TextView
    android:id="@+id/tvBody"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="14sp"
    android:textColor="@android:color/darker_gray"
    android:maxLines="3"
    android:ellipsize="end"
    android:layout_marginTop="4dp" />

<TextView
    android:id="@+id/tvUserId"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="12sp"
    android:textColor="@android:color/holo_blue_dark"
    android:layout_marginTop="4dp" />

</LinearLayout>

```

3.16 Жизненный цикл синхронизации данных

Процесс синхронизации данных между сетевым API и локальной базой происходит следующим образом:

- 1) Пользователь нажимает кнопку «Обновить» или выполняет pull-to-refresh.
- 2) ViewModel вызывает метод refreshMessages() в Repository.
- 3) Repository проверяет доступность сетевого соединения через NetworkUtils.
- 4) Если соединение доступно, Repository запрашивает данные через ApiService (Retrofit).
- 5) Полученные данные преобразуются из MessageResponse в сущность Message.
- 6) Локальная база очищается и пополняется новыми данными через MessageDao.
- 7) LiveData в MessageDao уведомляет наблюдателей об изменении (фрагмент обновляет RecyclerView).

8) При отсутствии соединения используются данные из локальной базы.

3.17 Результаты и выводы

По завершении третьей лабораторной работы была реализована полнофункциональная система загрузки данных с поддержкой сетевого кэширования. Использование паттерна Repository обеспечило чистое разделение ответственности между слоями приложения, а применение Room для локального хранилища позволило реализовать надёжный механизм кэширования и работу в режиме offline.

Интеграция Retrofit и Room с LiveData обеспечила реактивное обновление интерфейса при изменении данных, а использование корутин гарантировало асинхронность всех операций без блокировки основного потока. Механизм pull-to-refresh и кнопка ручного обновления предоставили пользователю удобные способы синхронизации данных.

4 Лабораторная работа 4

4.1 Задачи и требования

Четвёртая лабораторная работа была посвящена интеграции компонентов Material Design, реализации интерактивности пользовательского интерфейса (функция лайков) и организации фоновой синхронизации данных через WorkManager с поддержкой уведомлений. Требовалось:

- 1) Переработать интерфейс ленты с использованием Material Design компонентов (MaterialCardView, уточненные иконки).
- 2) Добавить функцию лайков с визуальной индикацией (иконка меняется при клике).
- 3) Реализовать фоновую синхронизацию через WorkManager с периодичностью 15 минут.
- 4) Настроить уведомления при успешной синхронизации.
- 5) Запросить runtime разрешения (уведомления, контакты).
- 6) Обеспечить корректное управление состоянием при offline/online режимах.

4.2 Обновлённая модель данных

К сущности Message добавлено поле isLiked для отслеживания статуса лайка пользователя.

Листинг кода – Message.kt:

```
@Entity(tableName = "messages")
data class Message(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0,
    val title: String,
    val body: String?,
    val userId: Int,
    var isLiked: Boolean = false
)
```

4.3 Адаптер с функцией лайков

Адаптер содержит логику переключения статуса лайка и обновления иконки.

Листинг кода – MessageAdapter.kt:

```
inner class MessageViewHolder(itemView: View) :
RecyclerView.ViewHolder(itemView) {
    private val ibLike: ImageButton =
itemView.findViewById(R.id.ibLike)

    fun bind(message: Message) {
        applyLikeIcon(message.isLiked)

        ibLike.setOnClickListener {
            message.isLiked = !message.isLiked
            applyLikeIcon(message.isLiked)
        }
    }

    private fun applyLikeIcon(isLiked: Boolean) {
        val iconRes = if (isLiked)
            R.drawable.ic_heart_filled
        else
            R.drawable.ic_heart_empty
        ibLike.setImageResource(iconRes)
    }
}
```

4.4 Material Design в разметке элемента

Элемент списка использует MaterialCardView с тенью и аватаром пользователя.

Листинг кода – item_message.xml (ключевые элементы):

```
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="12dp"
    android:elevation="4dp">

    <LinearLayout>
        <!-- Аватар -->
        <de.hdodenhof.circleimageview.CircleImageView
            android:id="@+id/ivAvatar"
            android:layout_width="40dp"
            android:layout_height="40dp"
            android:src="@drawable/ic_user_avatar" />
```

```

        <!-- Имя пользователя -->
        <TextView android:id="@+id/tvName" ... />

        <!-- Кнопка лайка -->
        <ImageButton
            android:id="@+id/ibLike"
            android:layout_width="40dp"
            android:layout_height="40dp"
            android:src="@drawable/ic_heart_empty"

            android:background="?attr/selectableItemBackgroundBorderless" />
        </LinearLayout>
        ...
    </com.google.android.material.card.MaterialCardView>

```

4.5 WorkManager и фоновая синхронизация

SyncWorker выполняет периодическое обновление данных в фоне.

Листинг кода – SyncWorker.kt:

```

class SyncWorker(
    appContext: Context,
    workerParams: WorkerParameters
) : CoroutineWorker(appContext, workerParams) {

    override suspend fun doWork(): Result {
        return try {

            (!NetworkUtils.isNetworkAvailable(applicationContext)) {
                Log.d("SyncWorker", "No internet -> retry")
                return Result.retry()
            }

            val messageDao =
                AppDatabase.getDatabase(applicationContext).messageDao()
            val apiService = RetrofitClient.apiService

            val apiMessages = apiService.getPosts()
            val messages = apiMessages.map {
                Message(
                    id = it.id.toLong(),
                    title = it.title,
                    body = it.body,
                    userId = it.userId,
                    isLiked = false
                )
            }

            messageDao.deleteAllMessages()
            messageDao.insertMessages(messages)

```

```

NotificationHelper.showSyncNotification(applicationContext)

        Result.success()
    } catch (e: Exception) {
        Log.e("SyncWorker", "Sync error: ${e.message}", e)
        Result.retry()
    }
}
}

```

4.6 Уведомления при синхронизации

NotificationHelper управляет созданием канала и отправкой уведомлений.

Листинг кода – NotificationHelper.kt:

```

object NotificationHelper {
    private const val CHANNEL_ID = "sync_channel"
    private const val NOTIFICATION_ID = 1001

    fun createChannel(context: Context) {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                CHANNEL_ID,
                "Sync notifications",
                NotificationManager.IMPORTANCE_DEFAULT
            ).apply {
                description = "Notifications about background
sync"
            }

            val manager = context.getSystemService(
                Context.NOTIFICATION_SERVICE
            ) as NotificationManager
            manager.createNotificationChannel(channel)
        }
    }

    fun showSyncNotification(context: Context) {
        if (Build.VERSION.SDK_INT >=
Build.VERSION_CODES.TIRAMISU) {
            val granted = ActivityCompat.checkSelfPermission(
                context,
                Manifest.permission.POST_NOTIFICATIONS
            ) == PackageManager.PERMISSION_GRANTED
            if (!granted) return
        }
    }
}

```

```

        val notification = NotificationCompat.Builder(context,
CHANNEL_ID)
            .setSmallIcon(android.R.drawable.stat_notify_sync)
            .setContentTitle("Синхронизация")
            .setContentText("Новые данные получены")
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .build()

NotificationManagerCompat.from(context).notify(NOTIFICATION_ID,
notification)
    }
}

```

4.7 Запрос разрешений и планирование WorkManager

В MainActivity инициализируются разрешения и фоновая синхронизация.

Листинг кода – MainActivity.kt:

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    // Инициализация навигации (как в Lab 1)
    val navHostFragment = supportFragmentManager
        .findFragmentById(R.id.nav_host_fragment) as
NavHostFragment
    val navController = navHostFragment.navController
        val bottomNav =
findViewById<BottomNavigationView>(R.id.bottom_nav)
    bottomNav.setupWithNavController(navController)

    // 1. Создание канала для уведомлений
    NotificationHelper.createChannel(this)

    // 2. Запрос runtime разрешений
    requestRuntimePermissions()

    // 3. Планирование периодической синхронизации
    scheduleBackgroundSync()
}

private fun requestRuntimePermissions() {
    val permissions = mutableListOf<String>()

    // Android 13+ требует явного разрешения на уведомления
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        permissions.add(Manifest.permission.POST_NOTIFICATIONS)
    }
}

```

```

        // Разрешение на доступ к контактам (как требует задание)
        permissions.add(Manifest.permission.READ_CONTACTS)

requestPermissionsLauncher.launch(permissions.toTypedArray())
}

private fun scheduleBackgroundSync() {
    val constraints = Constraints.Builder()
        .setRequiredNetworkType(NetworkType.CONNECTED)
        .build()

    val request = PeriodicWorkRequestBuilder<SyncWorker>(
        15, TimeUnit.MINUTES
    ).setConstraints(constraints).build()

    WorkManager.getInstance(this).enqueueUniquePeriodicWork(
        "messages_sync_work",
        ExistingPeriodicWorkPolicy.UPDATE,
        request
    )
}

```

4.8 Объявление разрешений в манифесте

Листинг кода – AndroidManifest.xml:

```

<uses-permission android:name="android.permission.INTERNET" />
<uses-permission
    android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission
    android:name="android.permission.POST_NOTIFICATIONS" />
<uses-permission android:name="android.permission.READ_CONTACTS"
/>

<service

    android:name="androidx.work.impl.foreground.SystemForegroundService"
    android:foregroundServiceType="dataSync"
    tools:node="merge" />

```

4.9 Поток выполнения синхронизации

- 1) WorkManager запускает SyncWorker каждые 15 минут (при наличии сети).
- 2) SyncWorker проверяет доступность интернета через NetworkUtils.
- 3) При наличии соединения загружаются данные через ApiService.
- 4) Данные сохраняются в локальную базу Room.
- 5) NotificationHelper отправляет уведомление об успехе.

6) При отсутствии сети результат `Result.retry()` – `WorkManager` повторит попытку позже.

4.10 Реактивность интерфейса

Фрагмент лины подписан на изменения в `ViewModel.messages` через `LiveData`. При изменении данных в базе (включая статус лайков) интерфейс автоматически обновляется без пересоздания фрагмента.

4.11 Результаты и выводы

По завершении четвертой лабораторной работы было реализовано полнофункциональное приложение-мессенджер с интерактивным интерфейсом согласно стандартам `Material Design`. Интеграция `WorkManager` обеспечила надежную фоновую синхронизацию данных независимо от состояния приложения, а система уведомлений дала пользователю визуальную обратную связь о проведенных операциях.

Использование `runtime` разрешений обеспечило совместимость с современными версиями `Android (13+)`, а реактивная архитектура на основе `LiveData` гарантировала консистентность состояния приложения при различных сценариях использования.

ЗАКЛЮЧЕНИЕ

В ходе четырех лабораторных работ была спроектирована и реализована полнофункциональная мобильная приложение-мессенджер для платформы Android на языке программирования Kotlin. На первом этапе была установлена базовая архитектурная основа приложения с реализацией многоэкранной навигации, использующей Android Navigation Component, что обеспечило масштабируемость и удобство в управлении переходами между экранами. Введение полного логирования жизненного цикла компонентов позволило глубоко понять механизмы взаимодействия между фрагментами и активностью, что явилось критическим для последующих этапов разработки.

На втором этапе было внедрено архитектурное решение MVVM с использованием компонента ViewModel и механизма реактивного обновления интерфейса через LiveData. Реализация AppViewModel с персистентным сохранением данных через SharedPreferences обеспечила автоматическое сохранение состояния при изменении конфигурации устройства. Система двойной темы (светлая и темная) была реализована с реактивным применением к элементам интерфейса, что позволило пользователям выбирать предпочитаемый режим без потери данных при повороте экрана.

Третий этап был посвящен интеграции сетевых операций и реализации локального кэширования данных. Паттерн Repository обеспечил единственную точку контакта между слоями приложения, облегчив тестирование и модификацию источников данных. Использование Retrofit для взаимодействия с RESTful API и фреймворка Room для структурированного локального хранилища позволило реализовать асинхронное выполнение операций через корутины Kotlin. Особое внимание было уделено offline-first стратегии: при отсутствии сетевого подключения приложение загружает данные из локального кэша, а при восстановлении соединения автоматически синхронизирует данные с сервером. Механизмы pull-to-refresh и ручного обновления предоставили пользователю удобные способы контроля над процессом загрузки данных.

На четвертом этапе приложение трансформировалось из базовой читалки в полноценный интерактивный продукт с фоновой поддержкой. Внедрение Material Design компонентов, включая MaterialCardView и циклические аватары, повысило эстетическую привлекательность интерфейса. Реализация интерактивной функции лайков с визуальной обратной связью (иконка сердца меняется при клике) продемонстрировала применение реактивных паттернов для моментального обновления состояния. Интеграция WorkManager обеспечила надежную фоновую синхронизацию данных с периодичностью 15 минут, работающую независимо от состояния приложения. Push-уведомления при успешной синхронизации дали пользователю визуальную обратную связь о проведенных операциях. Реализация runtime разрешений для уведомлений и доступа к контактам обеспечила совместимость с современными версиями Android (13+), а корректная обработка offline/online состояний сети гарантировала стабильность работы в различных сетевых условиях.

Архитектурные решения, принятые на протяжении разработки, обеспечили достижение ключевых целей. Трехслойная архитектура, разделяющая код на слой пользовательского интерфейса, слой логики приложения (ViewModel и Repository) и слой данных (Room, Retrofit, SharedPreferences), обеспечила тестируемость, модульность и масштабируемость. Использование LiveData вместо простых обратных вызовов позволило избежать утечек памяти благодаря автоматическому управлению жизненным циклом, реализовать автоматические обновления интерфейса при изменении данных и сохранить состояние при пересоздании фрагментов. Offline-first подход приоритизации локального кэша обеспечил немедленное отображение кэшированных данных при запуске, попытки обновления в фоне при доступности сети и непрерывное использование приложения в режиме offline.

Разработанное приложение демонстрирует профессиональный подход к архитектуре с применением MVVM и Repository паттернов,

соответствующих индустриальным стандартам. Оно охватывает полный жизненный цикл приложения от инициализации до фоновой работы, применяет современные практики Android разработки с использованием AndroidX библиотек, Kotlin и корутин. Интерактивность, реактивные обновления и плавные переходы обеспечивают позитивный пользовательский опыт, в то время как обработка ошибок, работа в условиях offline и graceful fallback'и гарантируют надёжность. Все операции корректно следуют жизненному циклу компонентов: корутины отменяются при уничтожении ViewModel, наблюдатели LiveData автоматически отписываются, а ресурсы (базы данных, сетевые соединения) освобождаются надлежащим образом. Приложение полностью поддерживает как светлую, так и тёмную тему с корректным применением цветов Material Design для обоих режимов, а также корректно запрашивает и обрабатывает runtime разрешения на разных версиях Android.

Последовательное применение архитектурных паттернов в сочетании с современными компонентами Android Framework обеспечило создание масштабируемого, надёжного и удобного в использовании приложения. Критическая роль в успешной реализации принадлежала выбору правильной архитектуры на начальном этапе разработки и постепенному расширению функциональности с сохранением принципов разделения ответственности, что обеспечило возможность добавления сложных функций без необходимости значительного рефакторинга существующего кода. Приложение готово к дальнейшему развитию и демонстрирует применение лучших практик индустрии разработки мобильных приложений для Android платформы.